

# Risk Analysis Report

## 1. Purpose and Target Application Scope

This report is the **risk analysis report for the application under test**, not for the AutoTestDesign tool itself.

The independent application under test is **MiniShop Checkout**, a compact e-commerce checkout prototype implemented in:

- target\_app/minishop\_checkout/

The formal application definition is documented in:

- final\_docs/12\_target\_application\_definition\_cn.md

Within this project:

- ARG-Test is the testing tool
- MiniShop Checkout is the application under test
- coupon\_discount\_engine is the selected major module of MiniShop Checkout used for the detailed execution document

The scope of this report therefore covers the main risk-bearing parts of MiniShop Checkout:

- promotion and coupon logic
- shipping fee and threshold logic
- tax and order-total calculation
- payment-card input validation
- pickup-station and recipient validation
- checkout orchestration across the above components

This report does **not** treat ARG-Test itself as the system under risk analysis.

## 2. MiniShop Checkout Context

MiniShop Checkout represents the decision-heavy portion of a small online checkout flow. It is intentionally compact, but it still contains enough validation, threshold, and rule-combination logic to justify systematic testing.

The main application components used in this report are:

| Application component  | Main responsibility                                                | Main code path                                                                      | Requirement basis                                |
|------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------|
| Cart and order preview | represent items and compute merchandise subtotal                   | target_app/minishop_checkout/models.py, target_app/minishop_checkout/checkout.py    | subtotal assumptions for later pricing decisions |
| Promotion service      | validate and apply coupon rules                                    | target_app/minishop_checkout/promotion.py, reference_impl/coupon_discount_engine.py | coupon_discount_engine                           |
| Shipping service       | calculate fee by zone, weight, method, and free-shipping threshold | target_app/minishop_checkout/shipping.py                                            | shipping_fee_calculator                          |
| Tax service            | compute taxable amount, tax,                                       | target_app/minishop_checkout/tax.py                                                 | order_total_tax_calculation                      |

| Application component      | Main responsibility                                                              | Main code path                                        | Requirement basis                                   |
|----------------------------|----------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------------|
|                            | and final order total                                                            |                                                       |                                                     |
| Payment validation service | validate cardholder name, card number, expiry, CVV, and brand rules              | <code>target_app/minishop_checkout/payment.py</code>  | <code>payment_card_expiry_and_cvv_validation</code> |
| Pickup validation service  | validate pickup station ID, recipient phone, pickup code, and note               | <code>target_app/minishop_checkout/pickup.py</code>   | <code>pickup_station_contact_validation</code>      |
| Checkout orchestration     | combine promotion, shipping, tax, payment, and pickup decisions into one preview | <code>target_app/minishop_checkout/checkout.py</code> | integration of the above requirement concerns       |

This component map is the basis for the risk register and the test-plan suite structure.

## 3. Risk Scoring Method

### 3.1 Scoring Dimensions

Each risk is evaluated with three 1-5 dimensions:

- **Impact**: business or customer-facing damage if the defect escapes
- **Likelihood**: probability of defects based on rule density, boundaries, and interactions
- **Detectability**: difficulty of finding the defect without systematic testing

The final score is:

`Risk Priority = Impact x Likelihood x Detectability`

### 3.2 Priority Bands

- **High**:  $\geq 60$
- **Medium**: 36-59
- **Low**:  $\leq 35$

### 3.3 Evidence Sources Used in This Report

This report relies on:

- target-application code: `target_app/minishop_checkout/`
- selected detailed module: `reference_impl/coupon_discount_engine.py`
- target-application smoke tests: `tests/test_minishop_checkout_smoke.py`
- selected detailed-module tests: `tests/test_coupon_discount_engine_blackbox.py`,  
`tests/test_coupon_discount_engine_whitebox.py`
- formal target-application definition: `final_docs/12_target_application_definition_cn.md`

The broader repository requirement corpus remains useful for evaluating **ARG-Test** as a tool, but it is not the core object of this risk register.

## 4. Requirement- and Code-Driven Risk Inventory

**MiniShop Checkout** contains five main requirement-driven decision areas:

1. promotion logic with thresholds, exclusivity, and validity checks
2. shipping logic with zone tiers, free-shipping thresholds, and express restrictions
3. tax logic with country-specific taxable bases and rounding
4. payment validation with multiple required fields and brand-specific CVV rules
5. pickup validation with format, conditional requiredness, and contactless rules

In addition, the checkout service introduces one integration-level concern:

6. cross-component consistency between subtotal, promotion result, shipping fee, tax amount, and final order total

These areas define the main risks below.

## 5. Risk Register

| Risk ID | Affected component / feature                           | Requirement basis                                                                                 | Main failure concern                                                                                  | I/L/D | Priority | Band   |
|---------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|-------|----------|--------|
| R1      | Promotion discount and coupon logic                    | <code>coupon_discount_engine</code>                                                               | wrong discount amount, illegal coupon acceptance, missed exclusivity rule, financial leakage          | 5/5/4 | 100      | High   |
| R2      | Shipping fee calculation and method eligibility        | <code>shipping_fee_calculator</code>                                                              | wrong shipping fee, incorrect free-shipping grant, express shipment accepted when overweight          | 4/4/4 | 64       | High   |
| R3      | Tax and order-total calculation                        | <code>order_total_tax_calculation</code>                                                          | wrong taxable base, wrong country rate, rounding defects, incorrect final total                       | 5/4/4 | 80       | High   |
| R4      | Payment-card input validation                          | <code>payment_card_expiry_and_cvv_validation</code>                                               | invalid payment data accepted or valid payment data rejected                                          | 5/4/4 | 80       | High   |
| R5      | Pickup-station and recipient validation                | <code>pickup_station_contact_validation</code>                                                    | invalid station ID, phone, or pickup code accepted; legitimate pickup blocked                         | 4/3/4 | 48       | Medium |
| R6      | Checkout orchestration and cross-component consistency | integration of promotion, shipping, tax, payment, and pickup behavior in <code>checkout.py</code> | incorrect total after discount/shipping/tax, dropped validation status, inconsistent checkout preview | 4/4/4 | 64       | High   |

## 6. Priority Interpretation

### 6.1 High-Priority Areas

The highest-priority parts of **MiniShop Checkout** are:

- **promotion logic**
  - because it has direct financial impact and several interacting rules
- **tax and order-total calculation**
  - because even a small formula mistake changes what the customer pays
- **payment validation**
  - because acceptance/rejection mistakes break the checkout path or allow bad input through
- **shipping and checkout orchestration**
  - because threshold errors and cross-component inconsistencies are easy to miss if only nominal paths are tested

6.2 Medium-Priority Areas

The pickup-validation area is still important, but it has a narrower blast radius than pricing or payment logic:

- pickup-station format validation
- recipient phone validation
- contactless pickup-code validation

It must still appear in the test plan, but it does not require the same depth of executable evidence as the coupon module.

7. Risk-Driven Test Focus

The risk register directly drives technique choice for **MiniShop Checkout**.

| Risk class                         | Preferred techniques                  | Reason                                                               |
|------------------------------------|---------------------------------------|----------------------------------------------------------------------|
| Coupon and promotion risk          | Decision Table + EP + BVA + white-box | multiple interacting rules, thresholds, and rejection causes         |
| Shipping and tax calculation risk  | BVA + Decision Table                  | threshold edges, tier selection, country/rate combinations, rounding |
| Payment and pickup validation risk | EP + BVA                              | required-field, format, and inclusive-range validation               |
| Checkout orchestration risk        | scenario-based integration checks     | needed to verify that module outputs remain consistent when combined |

The selected major module **coupon\_discount\_engine** remains the best detailed-execution anchor because it combines:

- high financial impact
- multiple boundaries
- interacting business rules
- concrete expected results
- a compact Python implementation already backed by strong **pytest**, coverage, and mutation evidence

8. Relation to ARG-Test

**ARG-Test** supports the testing of **MiniShop Checkout** by:

1. analyzing requirement text
2. assigning risk and test priority
3. generating auditable black-box suites
4. exporting structured artifacts for review

However, this report remains an **application-risk** report.

The correct interpretation is:

- risk scoring here prioritizes the testing effort for **MiniShop Checkout**
- the tool is the means used to design and organize the tests

- the selected coupon module provides the strongest detailed execution evidence inside the target application
- 

## 9. Honest Boundaries

The following boundaries should stay explicit:

- **MiniShop Checkout** is a compact prototype created for this final project, not a production deployment
- the application code is concrete, but only **coupon\_discount\_engine** currently has full black-box, white-box, coverage, and mutation evidence
- the other application components currently provide concrete structure and smoke-level executable support rather than equally deep white-box analysis
- the repository still contains a broader requirement corpus for tool evaluation; those extra scenarios should not be confused with the core definition of the chosen target application

These boundaries do not weaken the project as long as the final submission states them accurately.

---

## 10. Conclusion

**MiniShop Checkout** is a concrete checkout-oriented target application with the highest risk concentrated in:

- coupon and promotion logic
- shipping/tax calculation
- payment validation
- checkout orchestration

The final conclusion of this report is:

- the target application has multiple **High** priority areas that justify systematic black-box testing
- **coupon\_discount\_engine** is the strongest selected high-risk module for detailed execution evidence
- the later test plan must therefore allocate the most testing effort to coupon logic, pricing math, payment validation, and integration consistency

This interpretation aligns the risk analysis report with the assignment requirement that the report should target the **application under test**, not the AutoTestDesign tool itself.